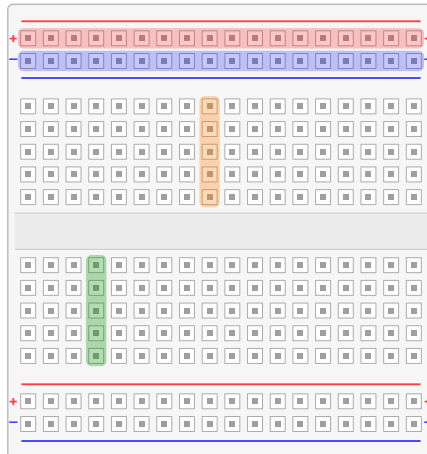


Protoboard & Arduino

como funcionam de verdade — em desenhos



O que tem aqui dentro

- Anatomia da protoboard: o que está ligado com o quê, e por quê
- O que existe embaixo do plástico (as molas de metal)
- Os 4 erros que fazem 90% dos circuitos não funcionarem
- Mapa do Arduino Uno peça por peça, com diagrama de blocos
- O que cada tipo de pino faz: digital, PWM, analógico, alimentação
- Três circuitos montados passo a passo: LED, botão e potenciômetro
- Cálculo do resistor, código de cores e checklist de depuração

1. A protoboard por fora: anatomia

Uma protoboard (ou *breadboard*) é um bloco de plástico furado com tiras de metal elástico por dentro. Cada tira liga um grupo de furos. Você não solda nada: enfia a perna do componente no furo, a mola aperta e o contato elétrico está feito.

Toda a dificuldade de aprender protoboard se resume a uma pergunta: **quais furos estão ligados entre si?**

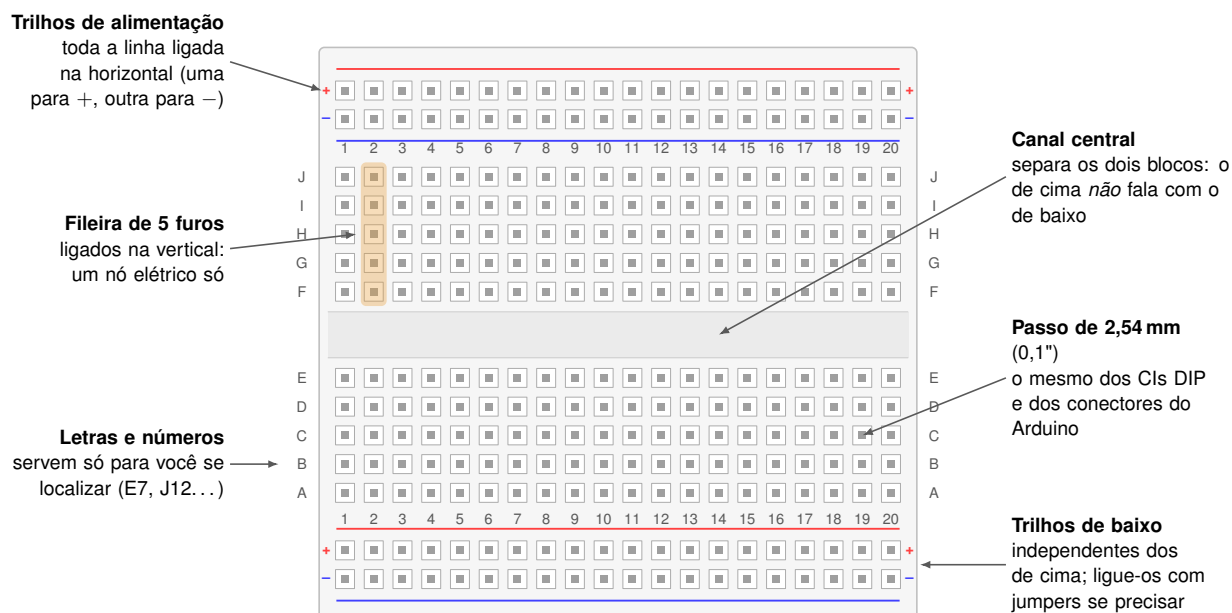


Figura 1 — protoboard de meia placa vista de cima (aqui com 20 colunas; as reais têm 30 ou 63).

2. A protoboard por dentro: as molas de metal

Se você levantar a fita adesiva de trás e tirar a capa de plástico, vai encontrar tiras de metal (bronze fosforoso niquelado, em geral) com pares de lâminas em forma de “V”. Cada tira serve um grupo de furos: é literalmente o fio que liga aqueles furos.

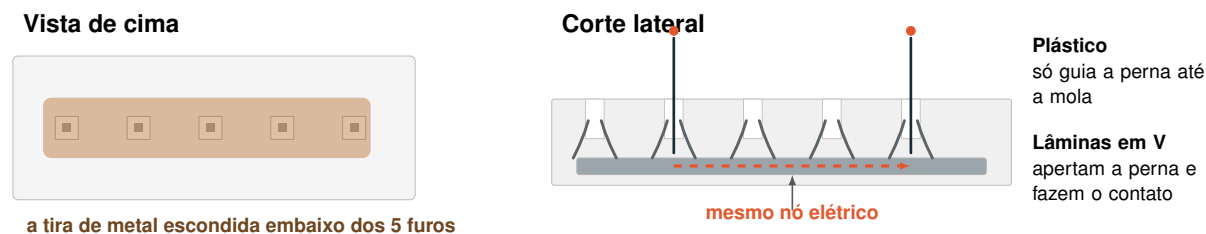


Figura 2 — o furo não é o contato; o contato é a lâmina de metal logo abaixo dele.

Consequências práticas disso (coisa de engenheiro)

- Cada contato tem uma **resistência** de dezenas de $m\Omega$ (e sobe com o uso). Por isso protoboard não gosta de corrente alta — fique abaixo de ~ 1 A por tira.
- Tiras vizinhas formam um **capacitor** de uns 2–5 pF. Acima de poucos MHz seu circuito começa a “ver” isso.
- Perna grossa demais (ex.: fio rígido $1,5\text{ mm}^2$ de instalação elétrica) **abre a mola** permanentemente. Use jumper 22 AWG ($\approx 0,64\text{ mm}$) ou perna de resistor.

3. O mapa de conexões (a única regra que importa)

Abaixo, a mesma placa com o metal “aparecendo” por transparência. Cada mancha de cor é uma tira de metal, ou seja, **um nó elétrico**.

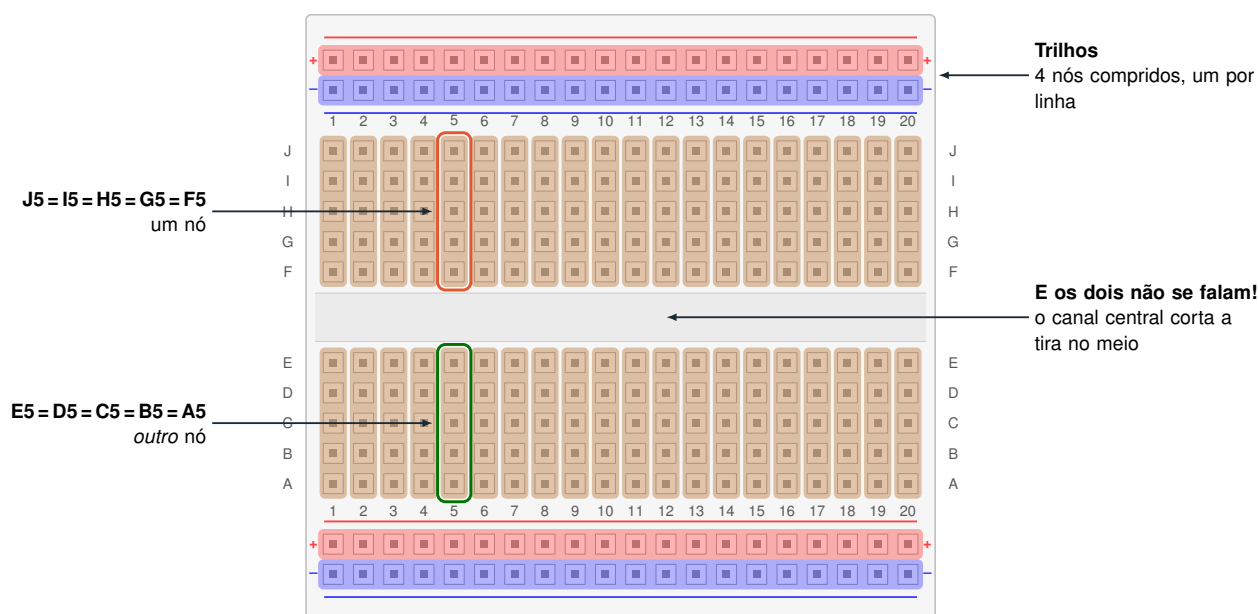


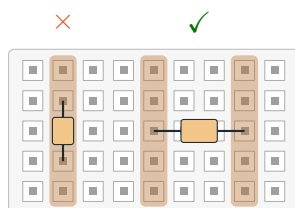
Figura 3 — “raio-x” da protoboard: cada mancha contínua é um fio.

Regra de ouro

Furos ligados pela mesma tira estão em **curto-circuito** entre si: mesmo potencial, sempre. Montar um circuito na protoboard é decidir *em qual nó* cada perna vai. Se duas pernas do mesmo componente caem na mesma tira, esse componente foi curto-circuitado e é como se não existisse.

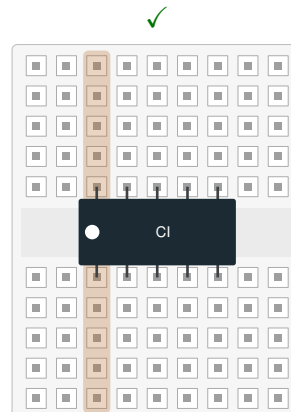
4. Os 4 erros que travam quase todo mundo

1. Componente “deitado” na mesma fileira



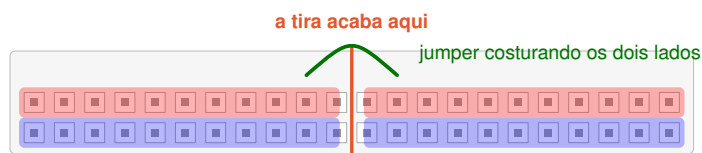
As duas pernas na mesma tira $\Rightarrow$ mesmo nó $\Rightarrow$ o resistor está em curto e não faz nada. Componente sempre **atravessa** de uma coluna para outra.

2. CI fora do canal central



Montado sobre o canal, cada pino cai numa coluna própria. Se você apoiar o CI inteiro de um lado só, as duas fileiras de pinos entram na mesma tira: curto em tudo.

3. Trilho de alimentação interrompido no meio



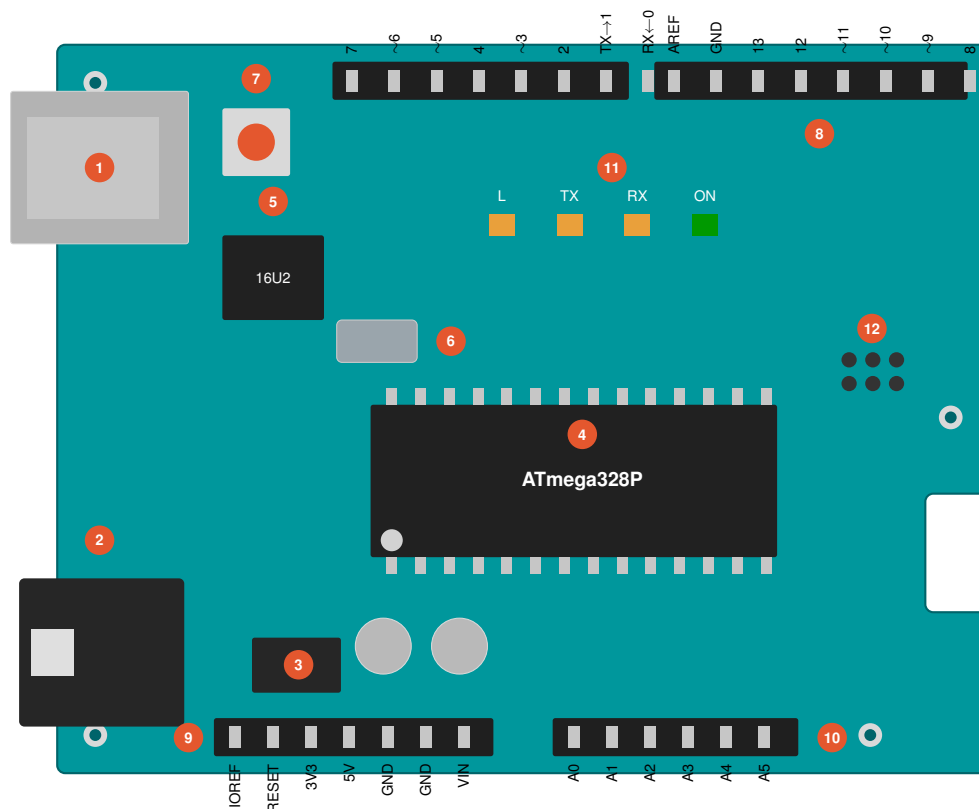
Muitas placas grandes cortam os trilhos ao meio — dá para perceber pelo pequeno espaço em branco na linha vermelha/azul impressa. Metade do circuito fica sem alimentação e você enlouquece procurando defeito. Teste com multímetro em continuidade antes de acusar o Arduino.

4. Esquecer o GND comum

Alimentou o circuito com uma fonte externa e leu o sinal com o Arduino, mas **não ligou o GND do Arduino ao negativo da fonte**? Então não existe referência comum: “5 V” não quer dizer nada se não for *em relação a alguma coisa*. Tensão é diferença de potencial. Sem GND compartilhado, leituras viram lixo e nada liga direito.

5. O Arduino Uno peça por peça

O Arduino não é um computador: é um **microcontrolador** (o chip ATmega328P) numa placa que já resolve para você a alimentação, o *clock*, o botão de reset e a conversão USB↔serial. Os conectores pretos nas bordas são só extensões dos pinos do chip.



1. USB tipo B — programa a placa e alimenta com 5 V (≤ 500 mA).

2. Jack de energia — 7 a 12 V DC, centro positivo.

3. Regulador linear — derruba a tensão do jack para 5 V queimando a diferença em calor.

4. ATmega328P — o cérebro: 32 kB de flash, 2 kB de RAM, 16 MHz.

5. ATmega16U2 — tradutor USB ↔ serial; sem ele o PC não conversa com o chip.

6. Cristal de 16 MHz — o metrônomo que dá o ritmo das instruções.

7. Botão de reset — reinicia o programa do zero.

8. Pinos digitais 0–13 — entrada ou saída, 0 ou 5 V. Os de til (~) fazem PWM.

9. Pinos de alimentação — 5V, 3V3, GND, VIN e RESET.

10. Entradas analógicas A0–A5 — leem tensão contínua entre 0 e 5 V.

11. LEDs — L ligado ao pino 13, TX/RX piscam na comunicação, ON é alimentação.

12. ICSP — gravação direta do chip, sem bootloader.

Figura 4 — desenho esquemático do Arduino Uno (proporções aproximadas).

6. Como a energia e os dados circulam por dentro

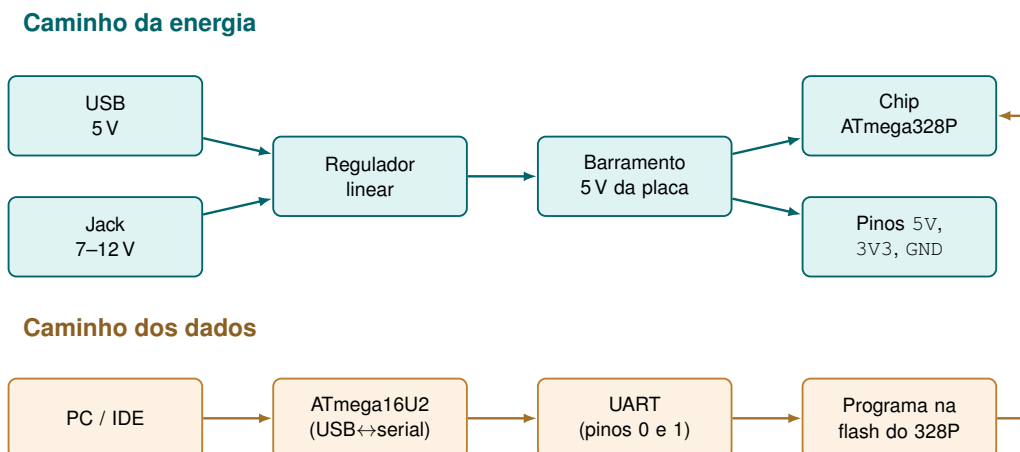


Figura 5 — energia entra pela esquerda; código entra por baixo. As duas coisas se encontram no chip.

Por que 7 a 12 V no jack, e não 20 V?

O regulador é *linear*: ele não converte energia, ele queima o excesso. Com 12 V na entrada e 200 mA de consumo, a potência dissipada é

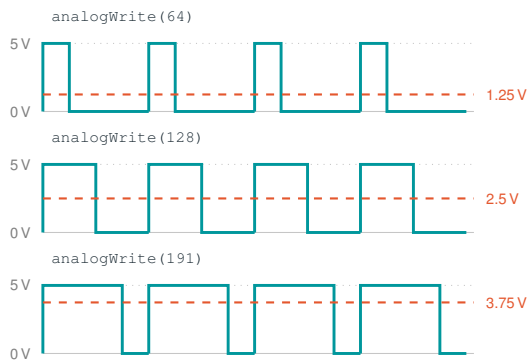
$$P = (V_{\text{in}} - V_{\text{out}})I = (12 - 5) \cdot 0,2 = 1,4 \text{ W},$$

tudo virando calor num encapsulamento minúsculo. Por isso 9 V é o valor mais confortável, e por isso alimentar motor pela placa é má ideia — motor pede fonte própria, com o GND compartilhado com o Arduino.

7. O que cada tipo de pino faz

Pino	Função	Detalhe que importa
0–13	Digital: só 0 V ou 5 V	Até 20 mA por pino com folga (40 mA é o limite absoluto), 200 mA somando tudo.
3 5 6 9 10 11	PWM (<code>analogWrite</code>)	Não é tensão analógica de verdade: é onda quadrada rápida, 0–255.
A0–A5	Entrada analógica	Conversor de 10 bits: devolve 0–1023. Também funcionam como digitais.
5V / 3V3	Saídas de alimentação	3V3 entrega só ~50 mA. 5V depende da fonte.
VIN	Entrada bruta	Ligada ao jack, antes do regulador.
GND	Referência (0 V)	Três pinos, todos o mesmo nó. É o que <i>tem</i> que ser comum a tudo.
0 e 1	RX e TX	Compartilhados com o USB: evite usar se for usar o <code>Serial</code> .

PWM: fingindo tensão intermediária

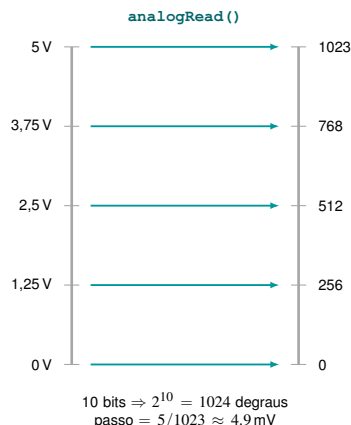


O pino só sabe 0 ou 5 V — ele alterna rápido (490 Hz ou 980 Hz) e o que o LED, o motor ou seu olho enxergam é a **média**.

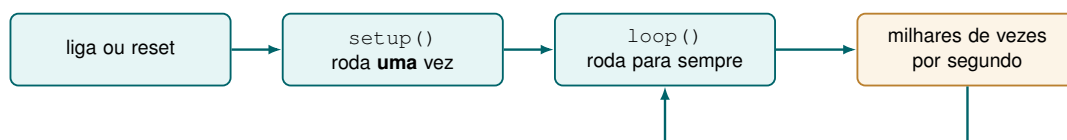
Para voltar de leitura para tensão:

$$V = \text{leitura} \cdot \frac{5,0}{1023} \quad \text{e, no PWM,} \quad \bar{V} = \frac{\text{valor}}{255} \cdot 5,0.$$

ADC: tensão vira número



8. O ciclo de vida de um programa



tudo que é configuração (`pinMode`, `Serial.begin`) vai no `setup`; tudo que é comportamento vai no `loop`

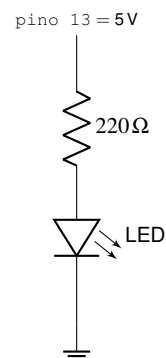
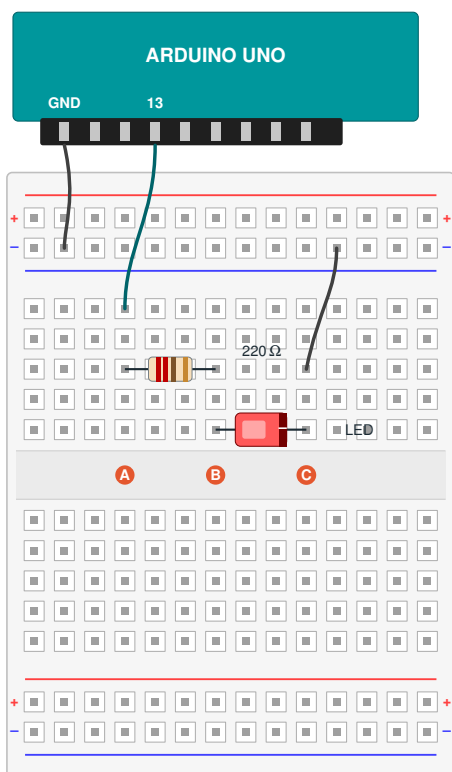
```

1 void setup() {           // executa 1 vez, logo apos ligar
2   pinMode(13, OUTPUT);   // "pino 13, voce vai EMPURRAR corrente"
3 }
4
5 void loop() {             // executa infinitamente, do inicio ao fim, sem parar
6   digitalWrite(13, HIGH); // coloca 5 V no pino 13
7   delay(500);             // trava tudo por 500 ms
8   digitalWrite(13, LOW);  // coloca 0 V no pino 13
9   delay(500);
10 }
```

pinMode não é burocracia

Um pino em OUTPUT é uma *fonte* de tensão: liga internamente o pino ao 5 V ou ao GND, com pouca resistência. Um pino em INPUT é quase um circuito aberto (alta impedância), feito para *observar* sem interferir. Ligar direto ao 5 V um pino que está em OUTPUT LOW é um curto: o chip morre.

9. Circuito 1: acender um LED de verdade



O LED é um diodo: ele *não* obedece à lei de Ohm e, ligado direto nos 5 V, puxa corrente até morrer (ou levar o pino junto). O resistor é quem define a corrente.

A: coluna 4 — tudo que sai do pino 13. B: coluna 7 — encontro do resistor com o anodo. C: coluna 10 — catodo indo para o trilho —, que vai ao **GND**.

Dimensionando o resistor — passo a passo

O LED vermelho cai cerca de $V_{LED} = 2,0 \text{ V}$ quando conduz. O resto da tensão sobra para o resistor:

$$V_R = V_{\text{pino}} - V_{LED} = 5,0 - 2,0 = 3,0 \text{ V.}$$

Escolhendo uma corrente confortável de $I = 15 \text{ mA}$ (o limite recomendado do pino é 20 mA):

$$R = \frac{V_R}{I} = \frac{3,0}{15 \times 10^{-3}} = 200 \, \Omega.$$

Não existe $200 \, \Omega$ na série comercial E12; o vizinho de cima é $220 \, \Omega$. Subindo o resistor, a corrente cai um pouco:

$$I_{\text{real}} = \frac{3,0}{220} = 13,6 \text{ mA} \quad (\text{seguro, e o brilho nem muda para o olho}).$$

Potência dissipada no resistor:

$$P = RI^2 = 220 \cdot (13,6 \times 10^{-3})^2 = 4,07 \times 10^{-2} \text{ W} = 40,7 \text{ mW},$$

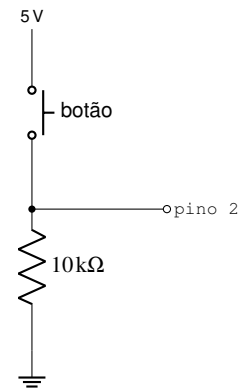
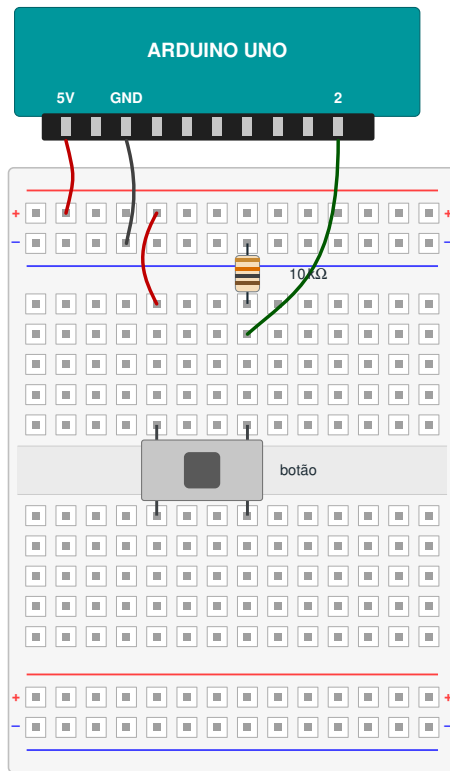
bem abaixo dos 250 mW de um resistor comum de $1/4 \text{ W}$. Pode usar.

```
1 const int LED = 13;           // nome melhor que numero solto
2
3 void setup() {
4     pinMode(LED, OUTPUT);
5 }
6
7 void loop() {
8     digitalWrite(LED, HIGH); delay(500);
9     digitalWrite(LED, LOW);  delay(500);
10 }
```


Como saber qual perna do LED é qual

A perna **maior** é o anodo (+, vai para o lado do resistor). Do lado do catodo (−) a saia do encapsulamento tem um **chanfro reto**. Se inverter, não queima nada — simplesmente não acende, o que já confundiu muita gente por horas.

10. Circuito 2: ler um botão (e o mistério do resistor de pull-down)



Solto: o pino enxerga o GND através dos $10\text{k}\Omega \Rightarrow$ lê LOW.

Pressionado: os 5 V chegam direto ao pino $\Rightarrow$ lê HIGH, e pelo resistor escorrem $5/10000 = 0,5\text{ mA}$ para o GND — desprezível.

Sem o resistor, a entrada “flutua”

Um pino em INPUT tem impedância altíssima. Sem o pull-down, quando o botão está solto o pino não está ligado a *nada*: vira uma antena de 60 Hz e de ruído do ambiente, e `digitalRead` devolve 0 e 1 alternados sem você encostar em nada. O resistor não é firula: é o que garante um valor definido no estado de repouso.

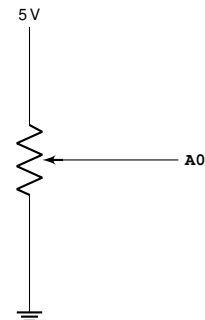
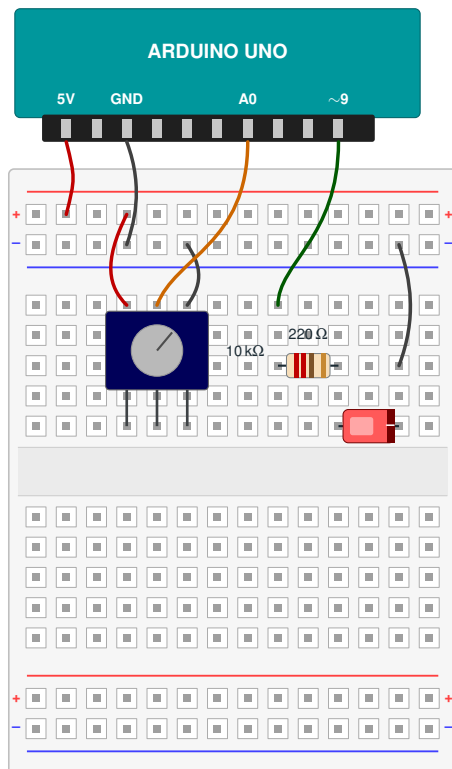
```
1  const int BTN = 2, LED = 13;    // 13 já tem
    LED na propria placa
2
3  void setup() {
4      pinMode(BTN, INPUT);
5      pinMode(LED, OUTPUT);
6  }
7
8  void loop() {
9      digitalWrite(LED, digitalRead(BTN));
10 }
```

Atalho: pull-up interno

O chip já tem resistores de $\sim 20\text{--}50\text{ k}\Omega$ ligados ao 5 V, desligados de fábrica. Com `pinMode(BTN, INPUT_PULLUP)` você liga um deles, joga o botão entre o pino e o GND e dispensa o resistor externo. O preço é a lógica invertida: solto lê HIGH, pressionado lê LOW.

11. Circuito 3: potenciômetro controlando o brilho

Este junta tudo: uma tensão analógica entra por **A0**, vira número, e sai como PWM no pino 9 para o mesmo LED do circuito 1.



O potenciômetro é um **divisor de tensão ajustável**: as duas pontas ficam em 5 V e 0 V, e o cursor entrega qualquer valor entre os dois. É por isso que ele precisa das *três* pernas ligadas — com duas, ele vira só um resistor variável.

```
1  const int POT = A0, LED = 9;    // 9 tem til: faz PWM
2
3  void setup() {
4      pinMode(LED, OUTPUT);
5      Serial.begin(9600);          // para espiar os valores no monitor serial
6  }
7
8  void loop() {
9      int leitura = analogRead(POT);          // 0 .. 1023 (10 bits)
10     int duty     = map(leitura, 0, 1023, 0, 255); // 0 .. 255 (8 bits do PWM)
11     analogWrite(LED, duty);
12     Serial.println(leitura);
13     delay(20);
14 }
```

Por que map () e não uma divisão qualquer

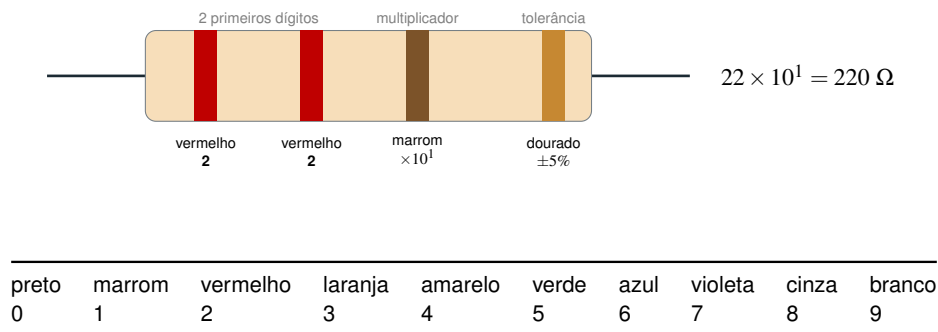
map(x, 0, 1023, 0, 255) calcula internamente

$$\text{duty} = \frac{x(255-0)}{1023-0} = \frac{255x}{1023} \approx \frac{x}{4,01},$$

com aritmética inteira. Se o cursor está no meio, $x \approx 512$ e o duty vira 127 — ou seja, cerca de 2,5 V médios no LED. É a cadeia completa: tensão → número → largura de pulso → brilho.

12. Colar de bolso

Código de cores do resistor



Quando nada funciona, nesta ordem

1. **O GND é comum?** Arduino, protoboard e qualquer fonte externa precisam compartilhar o negativo. É o erro número um, disparado.
2. **Os dois terminais do componente estão em colunas diferentes?** Se estão na mesma, ele está em curto.
3. **O trilho de alimentação está inteiro?** Meça continuidade de ponta a ponta.
4. **O LED está invertido?** Perna maior para o lado positivo.
5. **O pino do código é o pino do fio?** Contar a barra de pinos de trás para frente é clássico.
6. **pinMode foi declarado no setup?**
7. **O jumper está mesmo encaixado?** Puxe de leve cada um: os cansados soltam.
8. **Meça.** Multímetro em tensão DC, preta no GND, vermelha no ponto suspeito. Você para de adivinhar em dez segundos.

Onde a protoboard não serve

Situação	Por quê
Corrente acima de ~1 A	A resistência de contato aquece a mola e ela perde a força para sempre.
Sinais acima de poucos MHz	Capacitância entre tiras e indutância dos jumpers deformam o sinal.
Áudio sensível ou medida de precisão	Contatos oxidados injetam ruído e resistências imprevisíveis.
Qualquer coisa que vá sacudir ou viajar	Vibração solta os jumpers. Depois da prototipagem, migre para placa perfurada ou PCB.
Tensão de rede (127/220 V)	Nunca. O isolamento não foi feito para isso.

Resumo em três frases

A protoboard é um monte de fios escondidos: descobrir quais furos são o mesmo nó é 90% do trabalho. O Arduino é um chip com pinos que só sabem dois estados, mais alguns truques (PWM e conversor A/D) para fingir mundo analógico. O resto é você decidir em qual nó cada perna vai — e sempre, sempre, compartilhar o GND.